

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет програмних систем та прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

ЛАБОРАТОРНА РОБОТА №4

з дисципліни

“Автоматизоване проєктування КС”

Тема: «Побудова моделі мікропрограмного автомата»

Виконав: Ляшенко Д.Д.
Студент групи КВ-33

Київ 2026

План виконання лабораторної роботи:

1. За заданою ЛСА побудувати ГСА. Провести розмітку ГСА для автомата Мура. Побудувати граф автомата Мура.
2. Описати граф використовуючи State Diagram Editor, після чого згенерувати код мовою A-HDL.
3. Перевірити правильність роботи на часових діаграмах та за графом переходів.

Варіант завдання:

№	ЛСА
9	$U_n\ Z_1\uparrow^1 \downarrow^3 U_1 \downarrow^1 U_2\ Z_2 \uparrow^2 U_3 \downarrow^4 U_4\ Z_3 \uparrow^3 U_5 \downarrow^2 Z_4 \uparrow^4 U_k$

Хід виконання лабораторної роботи:

1. Аналіз ЛСА та побудова ГСА

Стрілка ↑ з номером i, що стоїть після логічної умови Zj, означає перехід до стрілки ↓ з тим самим номером при Zj = 1; при Zj = 0 виконується наступний за порядком елемент ЛСА. Стрілка ↓ позначає точку, до якої виконується перехід. Для заданої ЛСА пари стрілок такі:

Перехід	Умова (↑)	Точка входу (↓)
1	після Z1 (при Z1 = 1)	між U1 та U2 (перехід до U2)
2	після Z2 (при Z2 = 1)	між U5 та Z4 (перехід до перевірки Z4)
3	після Z3 (при Z3 = 1)	між Z1 та U1 (перехід до U1)
4	після Z4 (при Z4 = 1)	між U3 та U4 (перехід до U4)

За цією ЛСА побудовано ГСА та виконано розмітку для автомата Мура: початкова та кінцева вершини (Un, Uk) позначені станом q0, кожен оператор U1…U5 – станом q1…q5 (рисунок 1). Усього автомат має 6 станів, для кодування виходу достатньо 3 розрядів (Y[0:2]).

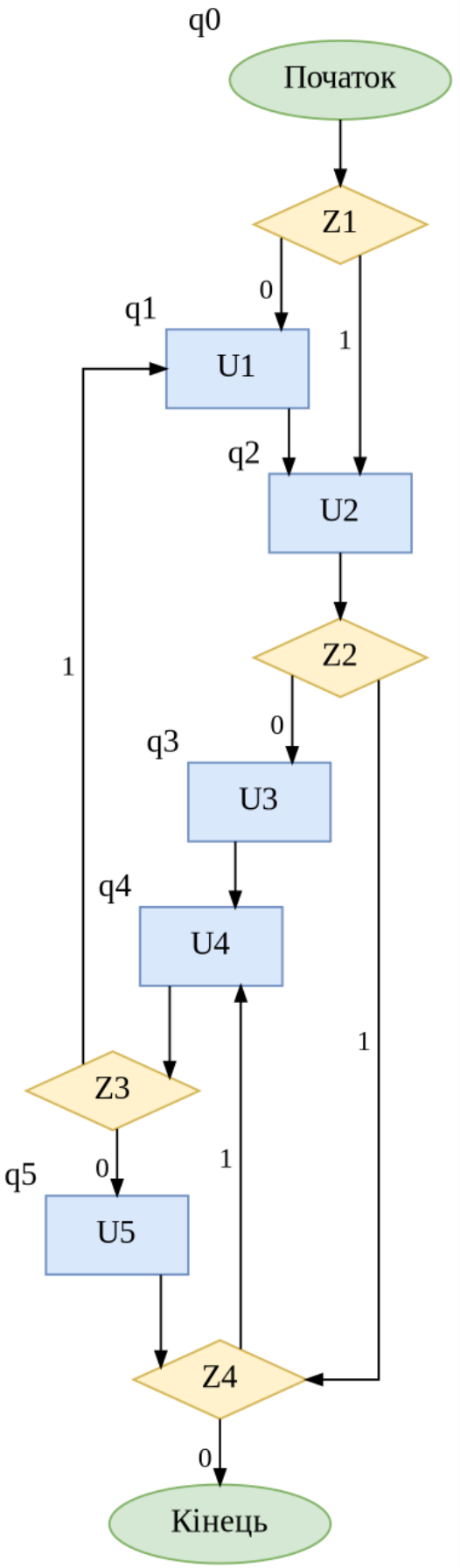


Рисунок 1 – ГСА з розміткою для автомата Мура

2. Таблиця переходів та граф автомата Мура

№	Зі стану	Умова переходу	У стан	Вихід Y
1	q0	Z1 = 0	q1	001
2	q0	Z1 = 1	q2	010
3	q1	–	q2	010
4	q2	Z2 = 0	q3	011
5	q2	Z2 = 1 та Z4 = 1	q4	100
6	q2	Z2 = 1 та Z4 = 0	q0	000
7	q3	–	q4	100
8	q4	Z3 = 1	q1	001
9	q4	Z3 = 0	q5	101
10	q5	Z4 = 1	q4	100
11	q5	Z4 = 0	q0	000

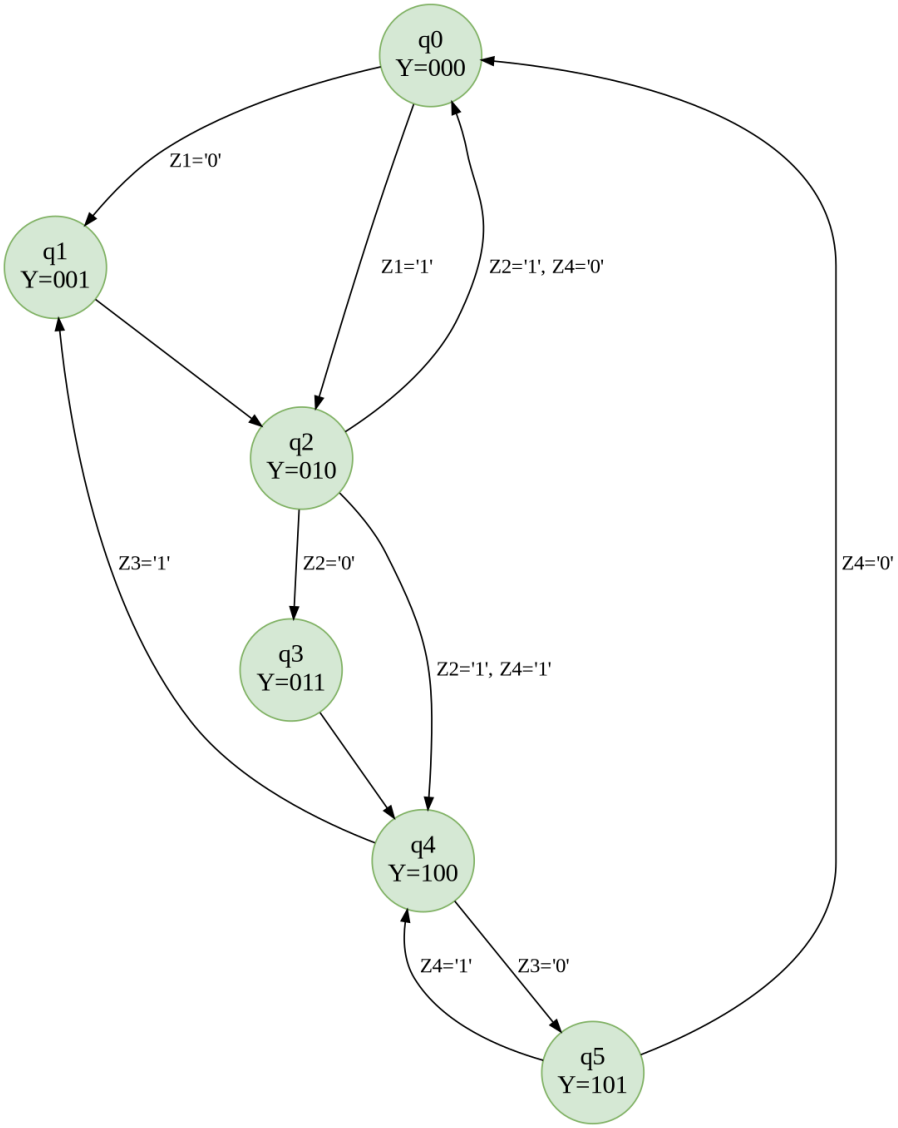


Рисунок 2 – Граф автомата Мура

3. Опис графа в State Diagram Editor

У середовищі Active-HDL створено файл State Diagram (мова генерації VHDL, ім’я lab4). Додано порти: вхід CLK, вхід Z[1:4] (умови Z1…Z4), вхід Reset та вихідну шину Y[0:2]. Створено вершини q0…q5, дуги з умовами переходів та дії станів Y<="…" (на виході видається номер вершини). У властивостях автомата початковим станом (скидання) встановлено q0 при Reset = ‘1’ (синхронне, активний високий рівень). Кінцевий вигляд графа наведено на рисунку 3.

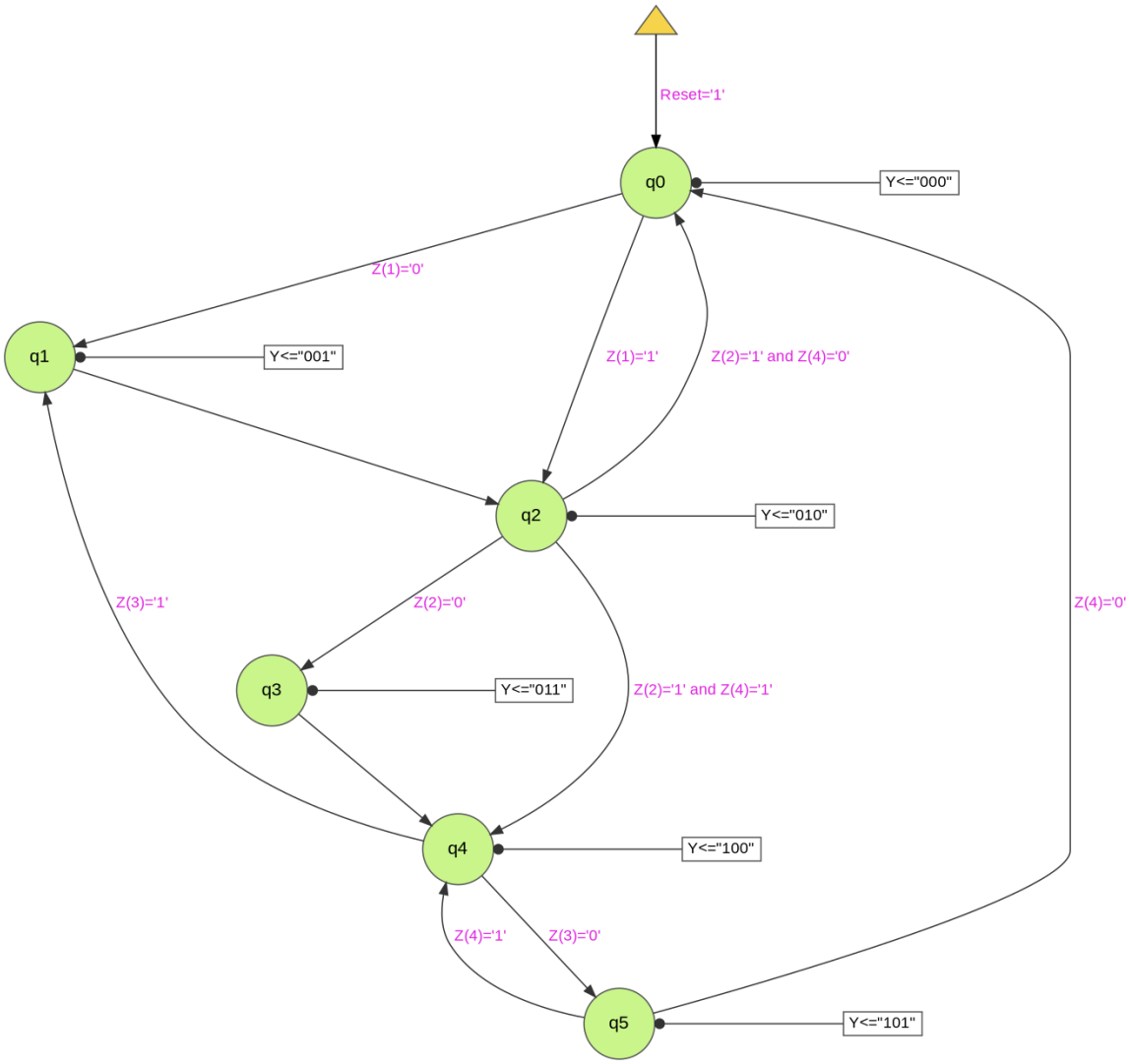


Рисунок 3 – Кінцевий вигляд графа в State Diagram Editor

LAB4 (код, згенерований за графом):

```
library IEEE;
use IEEE.std_logic_1164.all;

entity lab4 is
    port (
        CLK   : in  STD_LOGIC;
        Reset : in  STD_LOGIC;
        Z     : in  STD_LOGIC_VECTOR (1 to 4);   -- логічні умови Z1..Z4
```

```
        Y     : out STD_LOGIC_VECTOR (0 to 2)    -- код стану (номер вершини)
    );
end lab4;

architecture lab4 of lab4 is

-- стани автомата Мура
type Sreg0_type is (q0, q1, q2, q3, q4, q5);
signal Sreg0, NextState_Sreg0 : Sreg0_type;

begin

--------------------------------------------------------------------
-- регістр стану: синхронне скидання в q0 (Reset = '1')
--------------------------------------------------------------------
Sreg0_machine : process (CLK)
begin
    if CLK'event and CLK = '1' then
        if Reset = '1' then
            Sreg0 <= q0;
        else
            Sreg0 <= NextState_Sreg0;
        end if;
    end if;
end process;

--------------------------------------------------------------------
-- функція переходів
--------------------------------------------------------------------
Sreg0_next : process (Sreg0, Z)
begin
    NextState_Sreg0 <= Sreg0;
    case Sreg0 is
        when q0 =>
            if Z(1) = '0' then
                NextState_Sreg0 <= q1;
            elsif Z(1) = '1' then
                NextState_Sreg0 <= q2;
            end if;
        when q1 =>
            NextState_Sreg0 <= q2;
        when q2 =>
            if Z(2) = '0' then
                NextState_Sreg0 <= q3;
            elsif Z(2) = '1' and Z(4) = '1' then
                NextState_Sreg0 <= q4;
            elsif Z(2) = '1' and Z(4) = '0' then
                NextState_Sreg0 <= q0;
            end if;
        when q3 =>
            NextState_Sreg0 <= q4;
        when q4 =>
            if Z(3) = '1' then
                NextState_Sreg0 <= q1;
            elsif Z(3) = '0' then
                NextState_Sreg0 <= q5;
            end if;
        when q5 =>
            if Z(4) = '1' then
                NextState_Sreg0 <= q4;
            elsif Z(4) = '0' then
                NextState_Sreg0 <= q0;
            end if;
        when others => null;
    end case;
end process;

--------------------------------------------------------------------
-- функція виходів автомата Мура (залежить лише від стану)
--------------------------------------------------------------------
```

```
Sreg0_output : process (Sreg0)
begin
    case Sreg0 is
        when q0 => Y <= "000";
        when q1 => Y <= "001";
        when q2 => Y <= "010";
        when q3 => Y <= "011";
        when q4 => Y <= "100";
        when q5 => Y <= "101";
        when others => Y <= "000";
    end case;
end process;


end lab4;
```

TB_LAB4 (тестове оточення):

```
library IEEE;
use IEEE.std_logic_1164.all;


entity tb_lab4 is
end tb_lab4;


architecture tb of tb_lab4 is
    component lab4 is
        port (
            CLK   : in  STD_LOGIC;
            Reset : in  STD_LOGIC;
            Z     : in  STD_LOGIC_VECTOR (1 to 4);
            Y     : out STD_LOGIC_VECTOR (0 to 2)
        );
    end component;


    signal CLK   : STD_LOGIC := '0';
    signal Reset : STD_LOGIC := '1';
    signal Z     : STD_LOGIC_VECTOR (1 to 4) := "0000";
    signal Y     : STD_LOGIC_VECTOR (0 to 2);
begin
    UUT : lab4 port map (CLK, Reset, Z, Y);


    -- тактовий сигнал: період 10 ns (100 МГц), робочий фронт 5, 15, 25 ... ns
    clk_gen : process
    begin
        CLK <= '0'; wait for 5 ns;
        CLK <= '1'; wait for 5 ns;
    end process;


    stim : process
        procedure step(zv : STD_LOGIC_VECTOR(1 to 4);
                       expected : STD_LOGIC_VECTOR(0 to 2)) is
        begin
            Z <= zv;                            -- умови задаються до робочого фронту
            wait until rising_edge(CLK);
            wait for 1 ns;
            assert Y = expected
                report "Wrong state after transition"
                severity error;
        end procedure;
    begin
        Reset <= '1';
        wait until rising_edge(CLK);          -- синхронне скидання в q0
        wait for 1 ns;
        assert Y = "000" report "Reset: wrong state" severity error;
        Reset <= '0';
        step("1000", "010");   --  1: q0 -> q2   (Z1=1)
        step("0100", "000");   --  2: q2 -> q0   (Z2=1, Z4=0)
        step("0000", "001");   --  3: q0 -> q1   (Z1=0)
        step("0000", "010");   --  4: q1 -> q2
        step("0000", "011");   --  5: q2 -> q3   (Z2=0)
        step("0000", "100");   --  6: q3 -> q4
```

```
        step("0000", "101");    --  7: q4 -> q5    (Z3=0)
        step("0001", "100");    --  8: q5 -> q4    (Z4=1)
        step("0010", "001");    --  9: q4 -> q1    (Z3=1)
        step("0000", "010");    -- 10: q1 -> q2
        step("0101", "100");    -- 11: q2 -> q4    (Z2=1, Z4=1)
        step("0000", "101");    -- 12: q4 -> q5    (Z3=0)
        step("0000", "000");    -- 13: q5 -> q0    (Z4=0)
        wait for 20 ns;
        report "Simulation finished" severity note;
        wait;
    end process;
end tb;
```

Моделювання:

Для моделювання модулю lab4 встановлено статус Top-Level, виконано ініціалізацію моделювання та створено вікно Waveform із сигналами CLK, Reset, Z, Sreg0 (поточний стан) та Y. Тактовий сигнал має період 10 нс (100 МГц), робочий фронт – наростаючий. Значення Z змінюються між робочими фронтами. Тестова послідовність підібрана так, щоб пройти всі 11 переходів графа (таблиця нижче).

Загальна часова діаграма:

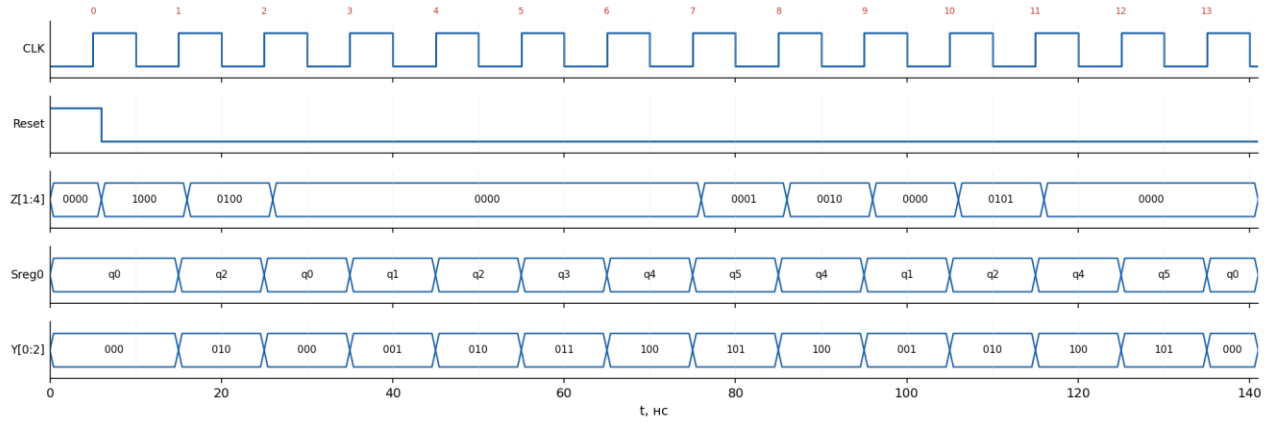


Рисунок 4 – Часова діаграма роботи автомата (цифрами позначено робочі фронти CLK)

На рисунку 5 показано умовний перехід q2 → q4 при Z2 = 1 та Z4 = 1 на фронті 115 нс: автомат переходить зі стану q2 (Y = 010) у стан q4 (Y = 100).

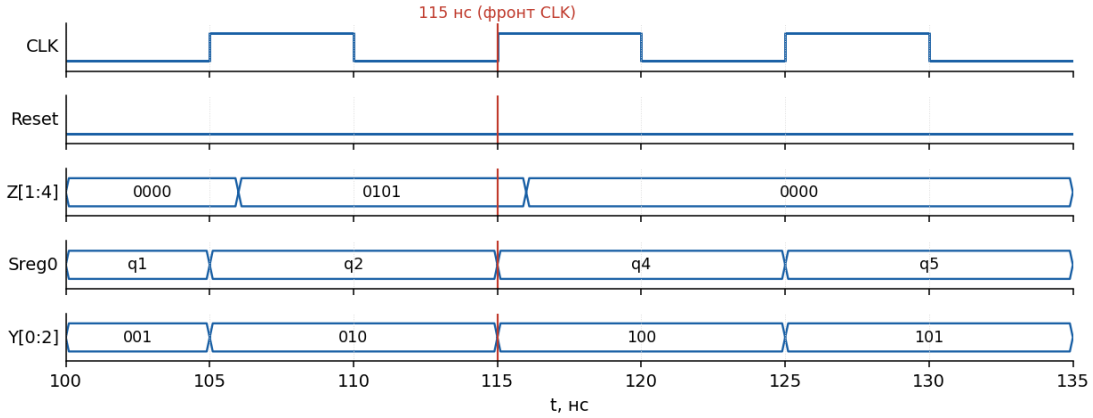


Рисунок 5 – Перехід q2 → q4 (Z2 = 1, Z4 = 1)

Результати перевірки переходів:

№	Фронт, нс	Z1Z2Z3Z4	Перехід	Очікуваний Y	Отриманий Y	Збіг
0	5	–	скидання → q0	000	000	✓
1	15	1000	q0 → q2	010	010	✓
2	25	0100	q2 → q0	000	000	✓
3	35	0000	q0 → q1	001	001	✓
4	45	0000	q1 → q2	010	010	✓
5	55	0000	q2 → q3	011	011	✓
6	65	0000	q3 → q4	100	100	✓
7	75	0000	q4 → q5	101	101	✓
8	85	0001	q5 → q4	100	100	✓
9	95	0010	q4 → q1	001	001	✓
10	105	0000	q1 → q2	010	010	✓
11	115	0101	q2 → q4	100	100	✓
12	125	0000	q4 → q5	101	101	✓
13	135	0000	q5 → q0	000	000	✓

Усі переходи збігаються з графом автомата Мура; перевірка виконана також автоматично за допомогою оператора assert у тестовому оточенні (повідомлень про помилки немає).

Висновок до лабораторної роботи:

У ході лабораторної роботи №4 в середовищі Active-HDL було побудовано та досліджено модель мікропрограмного автомата Мура за заданою ЛСА (варіант 9). За ЛСА побудовано ГСА, виконано її розмітку (початкова та кінцева вершини – стан q0, оператори U1…U5 – стани q1…q5) та складено таблицю з 11 переходів і граф автомата Мура з 6 станами та умовами Z1…Z4. Граф описано в State Diagram Editor, за ним згенеровано код мовою VHDL: регістр стану з синхронним скиданням у q0, функцію переходів та функцію виходів, яка видає на шину Y двійковий номер поточного стану. Під час моделювання підібрано тестову послідовність, що проходить усі переходи графа, зокрема умовні переходи з вершин q0, q2, q4 та q5. Часові діаграми та таблиця результатів показали, що автомат змінює стан лише по робочому фронту синхросигналу, значення виходу Y відповідає поточному стану (автомат Мура), а всі переходи збігаються з графом, наприклад q2 → q4 при Z2 = 1 і Z4 = 1 та q5 → q0 при Z4 = 0. Отже, побудований автомат працює правильно та відповідає заданій ЛСА, що підтверджує коректність виконання лабораторної роботи.